Switching the thimble code to the autocorrelator: exact changes, in two stages

Ross Jenkinson

16 July 2026

Why change observable

The propagator estimate is an *absolutely normalised* single integral, so it inherits every systematic of the estimator: the $(m/2\pi i a_c)^{N/2}$ prefactor, the branch and constant phase of $\det J$, and the contour orientation sign (the $\pm K$ flip we saw between seeds 42 and 43). Correlators are *ratios* of integrals over the same contour, so all of those cancel exactly. The physical target, $C(t) = \langle 0|\hat{x}(t)\hat{x}(0)|0\rangle = \sum_n |\langle 0|\hat{x}|n\rangle|^2 e^{-i(E_n - E_0)t}$, is also absolutely convergent with *no* η regulator, unlike K.

Do Stage 1 first: it needs no new training and no new contour, and it validates the machinery. Prerequisite for both stages: fix the Jacobian transpose bug first (`solve(A.t(), C)` → `solve(A, C)` in `compute_log_det_J`). The bug is z-dependent, so ratios do *not* cancel it.

Stage 1: pinned-endpoint correlators (zero new training)

Keep the current setup ($x_i = x_f = 0$, $T_c = T - i\eta$, same action, same trained model) and measure, for interior slices j, k,

$$G_{jk} \;=\; \langle x_j x_k \rangle \;=\; \frac{\int d^D x \; x_j x_k \, e^{iS}}{\int d^D x \; e^{iS}} \;=\; \frac{\mathbb{E}_z[\, x_j(z)\, x_k(z)\, W(z)\,]}{\mathbb{E}_z[\, W(z)\,]}, \qquad W = e^{\,iS + \log\det J - \log P}.$$

This is a reweighted observable over the samples you already draw. In the final Monte-Carlo block of `run_single`, after `log_W_mc` and `x_mc` are computed, add:

```
# G[j,k] = <x_j x_k>: ratio observable, needs NO norm_factor and NO exp(log_W_max)
W_sh   = torch.exp(log_W_mc - log_W_max)              # shifted weights (as now)
den    = torch.mean(W_sh)
G = torch.zeros((D_val, D_val), dtype=torch.complex64)
for j in range(D_val):
    G[j, :] = torch.mean(x_mc[:, j:j+1] * x_mc * W_sh.unsqueeze(1), dim=0) / den
# equal-time <x_k^2> is the diagonal; unequal-time correlator is G[j,k] vs (k-j)*a_c
```

Three things to note. First, do *not* multiply by `norm_factor` or `exp(log_W_max)`: both cancel in the ratio, which is the point. Second, $x_j(z)$ is complex on the deformed contour; that is correct, the insertion is analytically continued along with the contour. Third, error bars: use a bootstrap over samples of the ratio, not the naive std of the numerator, since numerator and denominator are correlated.

Exact benchmark. The same transfer matrix that gives the target K values gives G_{jk} exactly: propagate the chain from the left, multiply the running function by x at slice j and again at slice k, finish the chain, and divide by the plain chain. One-line change to the `lattice_K_transfer` function in my notebook (§16):

```
def lattice_G_transfer(lam, N_lat, j, k, **kw):     # 1 <= j <= k <= N_lat-1
    ...same setup as lattice_K_transfer...
    num = pref * np.exp(1j * m * (x - x_i)**2 / (2 * a_c))
    den = num.copy()
    for s in range(1, N_lat):                        # interior slices
        num_w = w * num
        if s == j: num_w = x * num_w
        if s == k: num_w = x * num_w                 # j == k handled twice: x^2
        num = h * _fftconv_valid(link, num_w)
```

```
        den = h * _fftconv_valid(link, w * den)
    Kn = np.interp(x_f, x, num.real) + 1j * np.interp(x_f, x, num.imag)
    Kd = np.interp(x_f, x, den.real) + 1j * np.interp(x_f, x, den.imag)
    return Kn / Kd
```

Acceptance test: your ML G_{jk} at $N = 32$ matches this to within bootstrap errors, for several (j, k) and $\lambda \in \{0, 0.5, 2.4\}$, across seeds, with *no* sign flips between seeds. Only then move to Stage 2.

Stage 2: the physical autocorrelator $C(t)$

The pinned-endpoint G_{jk} is a legitimate lattice observable but not $\langle 0|\hat{x}(t)\hat{x}(0)|0\rangle$: the endpoints inject excited states. The standard fix is a contour with **Euclidean end-caps**, which project onto the ground state:

$$z: \; 0 \xrightarrow{-i\tau_E} -i\tau_E \xrightarrow{t} t - i\tau_E \xrightarrow{-i\tau_E} t - 2i\tau_E,$$

i.e. N_E links with pure imaginary step $a = -i\delta$ ($\delta = \tau_E/N_E$), then N_R real-time links $a = t/N_R$ (keep a small $-i\eta/N_R$ tilt per real link for safety), then N_E more Euclidean links. Insert $\hat{x}$ at the two junction slices. Endpoints stay pinned at $x_i = x_f = 0$; the caps suppress excited states by $e^{-\tau_E(E_n - E_0)}$, so the projection error is $O(e^{-2\tau_E})$ for $\omega = 1$. $\tau_E \sim 3$ is plenty. Then

$$C(t) \;\simeq\; \frac{\int d^D x \; x_{j_2} x_{j_1} \, e^{iS}}{\int d^D x \; e^{iS}} \qquad (j_1, j_2 = \text{junction slices}),$$

again a ratio, so again no prefactor, branch, or orientation issues.

Concrete code changes, all small:

1. **Per-link steps.** Promote the scalar a_c to a length-N complex vector `a_vec`. In `complex_action`:

   ```
   kinetic  = (m/2) * torch.sum(dx**2 / a_vec, dim=1)          # a_vec: (N,) complex
   pot      = torch.sum(a_vec[1:] * V(x), dim=1)                # V on interior slices
   ```

   (check the slice alignment against your current convention once, at $\lambda = 0$, versus the vector-step lattice Gaussian, exactly as in the K validation).

2. **Normalisation.** $(m/2\pi i a_c)^{N/2} \to \prod_k (m/2\pi i a_k)^{1/2}$. It cancels in the ratio anyway; keep it only if you still want to quote the denominator as a propagator.

3. **Training.** Unchanged: same model, same variance loss, same $\log W$; only the action and the observable change. Train once on the capped contour per (λ, t).

4. **Benchmark.** The transfer matrix generalises immediately: use a per-link kernel with step a_k (Euclidean links get a real Gaussian kernel, real links the oscillatory one) and the same two x insertions. I will supply this; it is ten lines. Independently, my spectral code gives the exact answer: $C(t) = \sum_n |\langle 0|\hat{x}|n\rangle|^2 e^{-i(E_n - E_0)t}$ with $\hat{x}$ tridiagonal in the HO basis, so we get a continuum curve to sit alongside the lattice targets.

What to expect. The sign problem on the capped contour is milder than for the bare propagator at the same t, because the caps damp the large-field region; published thimble studies of exactly this observable (Alexandru et al., anharmonic oscillator real-time correlators) reached $t \sim$ a few in $\omega = 1$ units, so it is the right difficulty class for the method. The interesting scientific question, and the natural plot for the paper, is ESS versus t at fixed compute, ML contour versus flat tilted contour.

Checklist

1) Fix the transpose bug. 2) Reproduce the K targets at $N = 32$ (table in Ross's notebook §16, `lattice_targets_for_tom.csv`). 3) Stage 1: G_{jk} ratios on existing runs, benchmarked against `lattice_G_transfer`, stable across seeds. 4) Stage 2: capped contour, per-link a_k, measure $C(t)$, benchmark against the capped transfer matrix and the spectral curve.